# Professional Development

W07 Final Project | Open Gallery

| | |
|---|---|
| **STUDENT** | **OGWAL ALLAN** |
| **PROJECT** | Open Gallery - shared art search for the AIC and The Met |
| **LIVE APP** | https://allan2468coder.github.io/wdd330-sleepoutside/open-gallery/ |
| **SOURCE** | GitHub source folder |
| **TRELLO** | Smookie - Open Gallery project board |

## Project development summary

I developed Open Gallery as a responsive art-discovery experience within the existing Vite project. The application brings together public collection records from the Art Institute of Chicago (AIC) and The Metropolitan Museum of Art (The Met), allowing visitors to search either collection or both from one interface. Search results are generated from live JSON responses and normalized so the same result, details, and saved-work features can be used across the providers.

The work expanded from the initial search and results experience to include artwork detail pages, museum record links, saved works in browser local storage, responsive layouts, accessible interaction states, and production deployment through GitHub Pages. The interface communicates loading, empty results, partial provider failure, and unavailable records instead of leaving users without feedback.

## Professional skills practiced

### API research and integration

I compared the AIC and Met API structures and selected endpoints for both collection search and individual record details. Since the museums use different field names and image conventions, I convert their responses into a shared artwork format before rendering them. Met search results can require multiple object lookups, so I bound the concurrency of those requests to five at a time.

**Evidence:** artworks.js - API requests, normalization, and bounded Met requests.

### Maintainable frontend architecture

I separated API/data work, search page behavior, detail behavior, favorites persistence, markup, and CSS into focused modules and pages. This keeps changes local to the responsibility being updated and uses native browser APIs and ES modules rather than a JavaScript framework.

**Evidence:** main.js, artworks.js, detail.js, favorites.js, and favorites-page.js.

### Resilience and quality assurance

I handle the museums independently so a failure at one provider does not hide a successful response from the other. I added image fallbacks and meaningful status states. Automated tests cover normalization, pagination, Met request concurrency, API failures, and saved-work behavior. The project lint and production builds were also run.

**Evidence:** OpenGallery.test.js; npm test; npm run lint; standard and GitHub Pages builds.

### Accessibility and responsive interaction

I used semantic regions and labels, a skip link, visible keyboard focus, accessible status messages, alternative text, responsive layouts, and reduced-motion handling. Entrance animations are disabled when reduced motion is requested.

**Evidence:** The three Open Gallery HTML pages and styles.css.

### Deployment and project coordination

I configured a GitHub Actions workflow to build the Vite dist directory and publish it to GitHub Pages. I used the project Trello board to organize and report the W5, W6, and W7 tasks.

**Evidence:** deploy-pages.yml; public app and source links above; Smookie Trello board.

## Reflection

A key design decision was to normalize external API data at the application boundary. Although the museums expose different JSON schemas, the interface can work consistently when every result uses the same fields. I also learned that graceful network failure is part of frontend design: provider requests should be independent, and the page should explain which data is available rather than silently failing.

Testing revealed issues that a successful production build alone would not catch: the Jest setup referenced a missing Babel preset, and an empty test file caused the test command to fail before useful assertions ran. I corrected the test setup and added focused coverage for the app's data and persistence behaviors. This made the work more repeatable and gave me concrete checks for the important paths.

## Remaining student steps

• Confirm that the displayed name, OGWAL ALLAN, matches the required name order and spelling.

• Record and publish the required approximately five-minute screen demonstration with my face visible at the beginning; discuss the running application and the code.

• Replace the video placeholder below with the public recording URL, then test the link in a private/incognito browser.

• Review the official course template requirements and upload the required documents to Canvas.

**Video URL: [Add public video URL after recording]**

This PDF is a generated submission draft based on the project evidence. Review the first-person reflection for accuracy and complete the video field before final submission.